AI Assisted Coding Assignment- 13

RUDROJU RUPA SRI

2303A51918

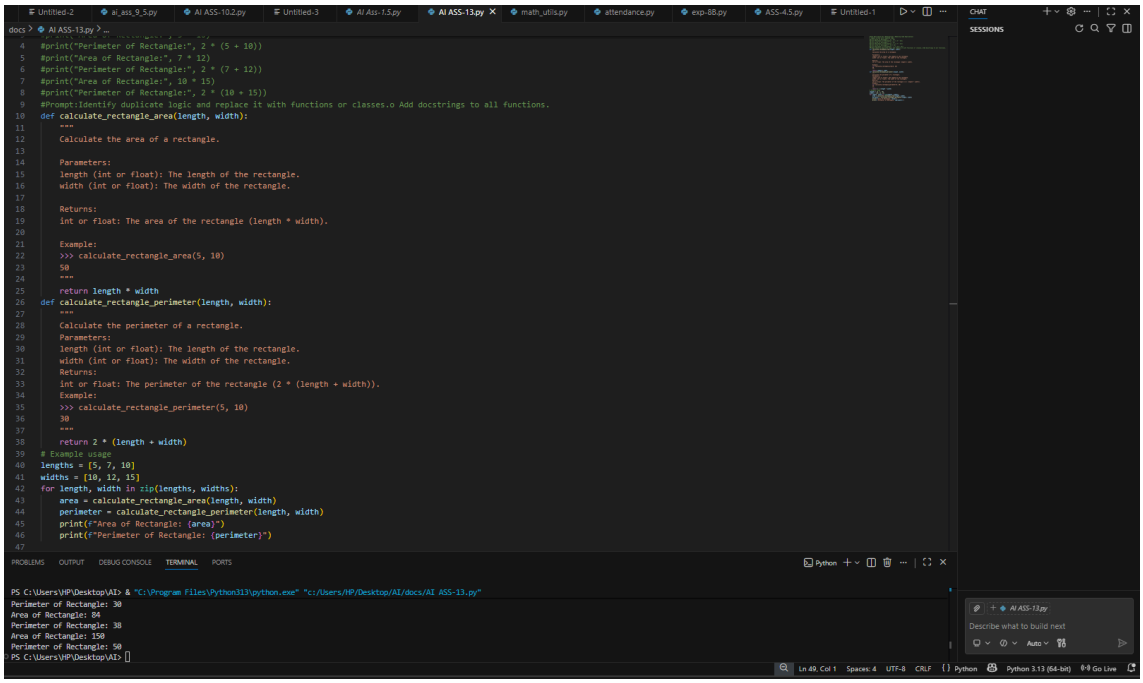
BATCH-30

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions
Lab Objectives:
Identify code smells and inefficiencies in legacy Python scripts.
Use AI-assisted coding tools to refactor for readability, maintainability, and performance.
Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)
Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
Instructions:
Prompt AI to identify duplicate logic and replace it with functions or classes.
Ensure the refactored code maintains the same output.
Add docstrings to all functions.
Sample Legacy Code:

```
# Legacy script with repeated logic

print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

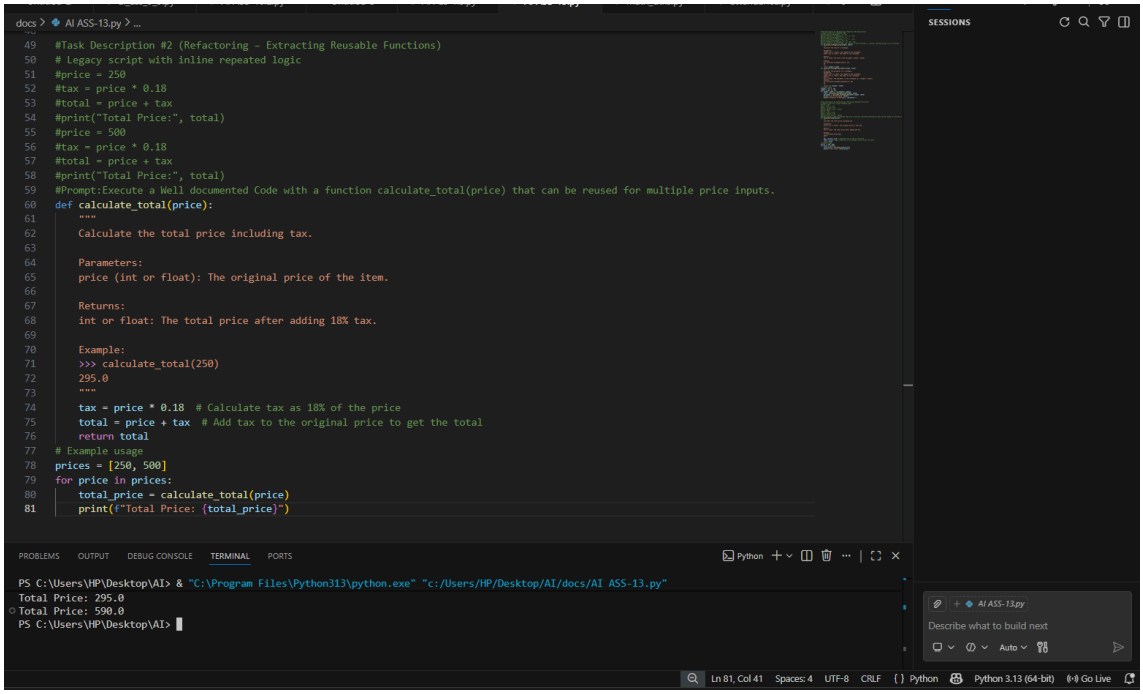
Expected Output:
Refactored code with a reusable function and no duplication.
Well documented code



Task Description #2 (Refactoring – Extracting Reusable Functions)
Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
Instructions:
Identify repeated or related logic and extract it into reusable functions.
Ensure the refactored code is modular, easy to read, and documented with docstrings.
Sample Legacy Code:

```
# Legacy script with inline repeated logic

price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Expected Output:
Code with a function calculate_total(price) that can be reused for multiple price inputs.
Well documented code



Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

Class Name: GradeCalculator

Method Name: calculate_grade(self, marks)

The method must:

Accept marks as a parameter.

Return the corresponding grade as a string.

The grading logic must strictly follow the conditions below:

Marks ≥ 90 and ≤ 100 → "Grade A"

Marks ≥ 80 → "Grade B"

Marks ≥ 70 → "Grade C"

Marks ≥ 40 → "Grade D"

Marks ≥ 0 → "Fail"

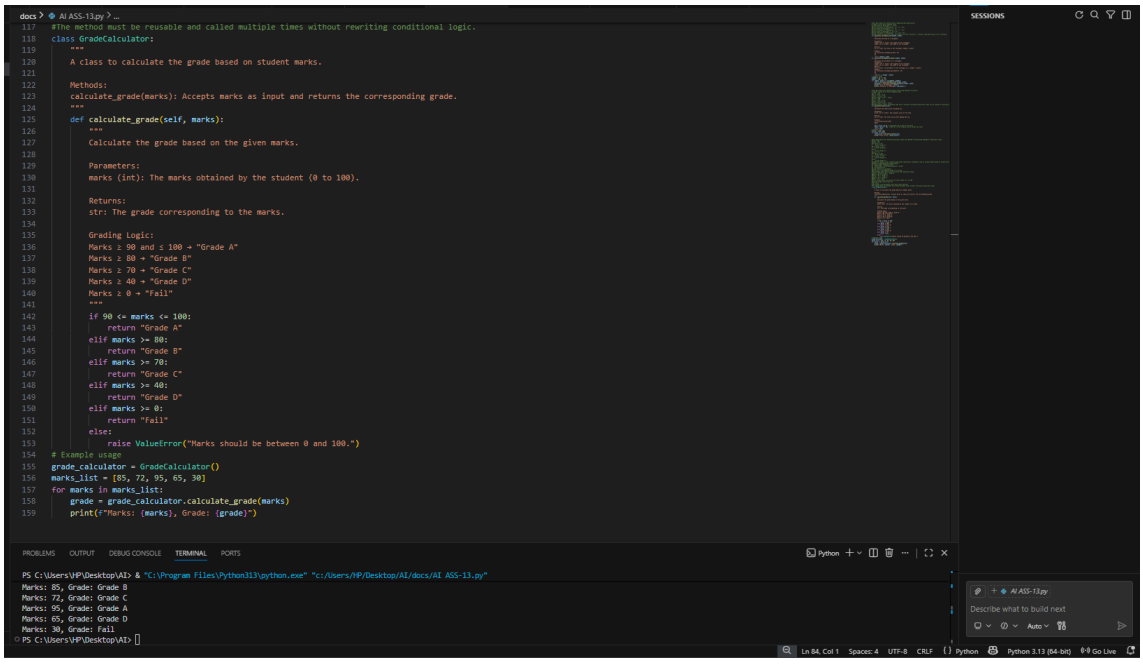
Note: Assume marks are within the valid range of 0 to 100.

Include proper docstrings for:

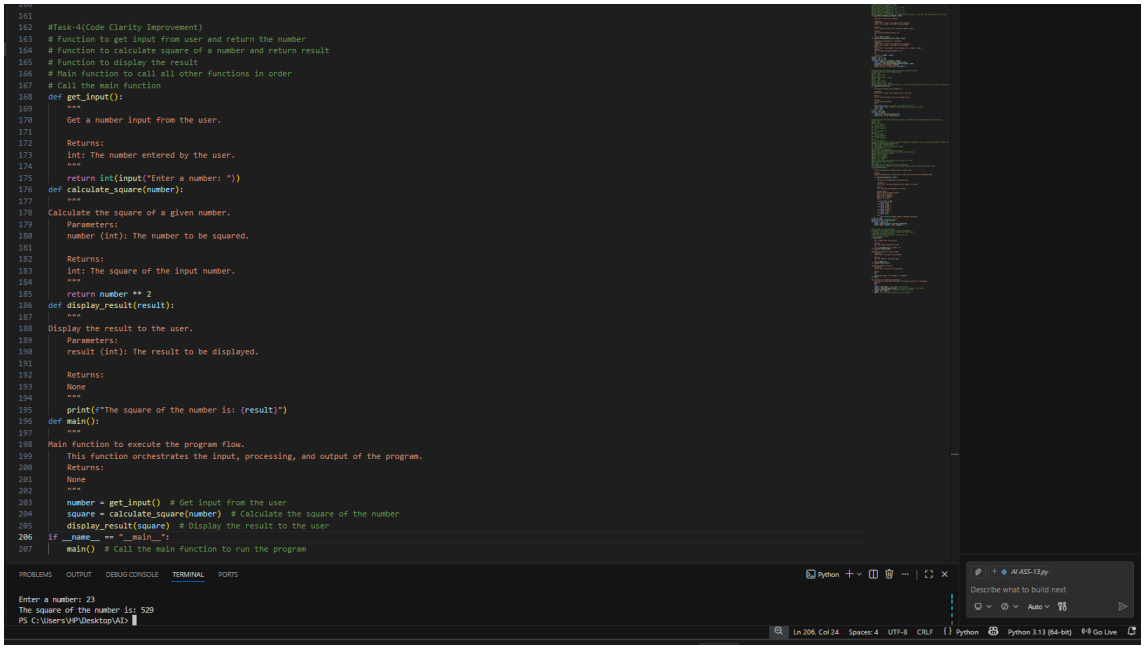
The class
The method (with parameter and return descriptions)
The method must be reusable and called multiple times without rewriting conditional logic.
• Given code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:
Define a class named GradeCalculator.
Implement a method calculate_grade(self, marks) inside the class.
Create an object of the class.
Call the method for different student marks.
Print the returned grade values.

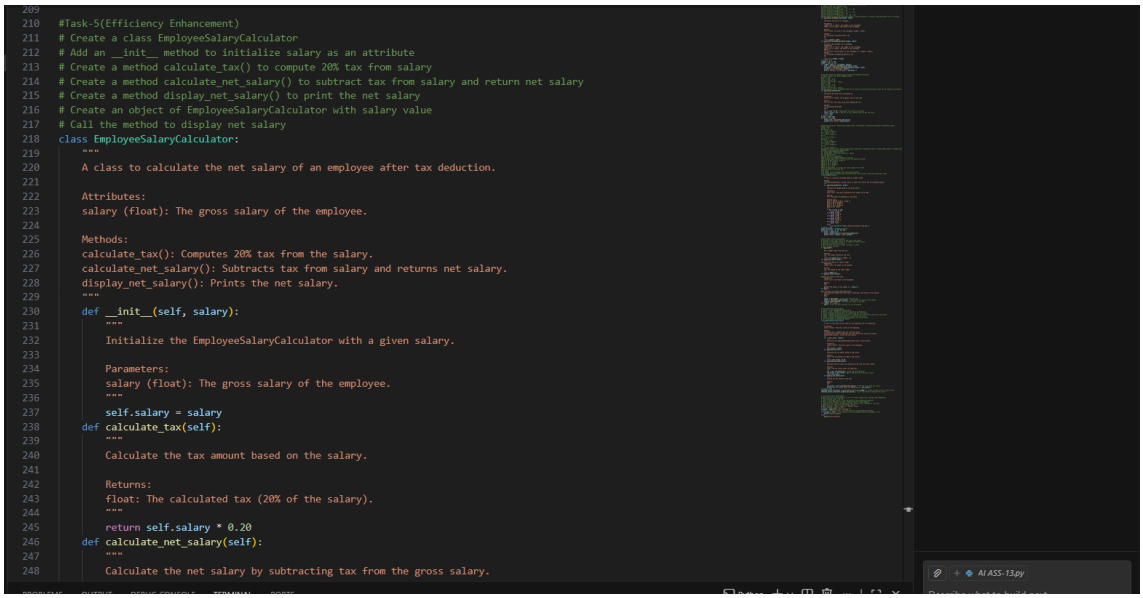


Task Description #4 (Refactoring – Converting Procedural Code to Functions)
• Task: Use AI to refactor procedural input–processing logic into functions.
Instructions:
o Identify input, processing, and output sections.
o Convert each into a separate function.
o Improve code readability without changing behavior.
• Sample Legacy Code:
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
• Expected Output:
o Modular code using functions like get_input(), calculate_square(), and display_result().

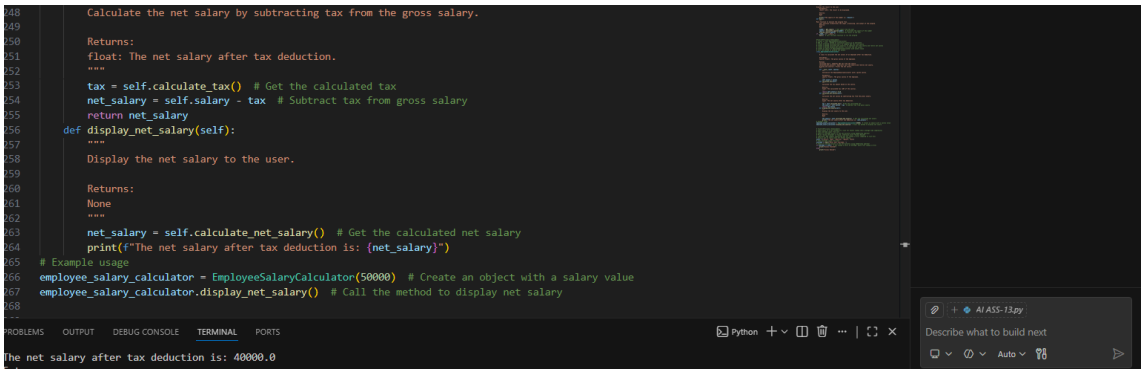


Task 5 (Refactoring Procedural Code into OOP Design)
• Task: Use AI to refactor procedural code into a class-based design.
Focus Areas:
o Object-Oriented principles
o Encapsulation
Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)

Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and attributes.



```python
#Task-5(Efficiency Enhancement)
# Create a class EmployeeSalaryCalculator
# Add an __init__ method to initialize salary as an attribute
# Create a method calculate_tax() to compute 20% tax from salary
# Create a method calculate_net_salary() to subtract tax from salary and return net salary
# Create a method display_net_salary() to print the net salary
# Create an object of EmployeeSalaryCalculator with salary value
# Call the method to display net salary
class EmployeeSalaryCalculator:
    """
    A class to calculate the net salary of an employee after tax deduction.

    Attributes:
    salary (float): The gross salary of the employee.

    Methods:
    calculate_tax(): Computes 20% tax from the salary.
    calculate_net_salary(): Subtracts tax from salary and returns net salary.
    display_net_salary(): Prints the net salary.
    """
    def __init__(self, salary):
        """
        Initialize the EmployeeSalaryCalculator with a given salary.

        Parameters:
        salary (float): The gross salary of the employee.
        """
        self.salary = salary
    def calculate_tax(self):
        """
        Calculate the tax amount based on the salary.

        Returns:
        float: The calculated tax (20% of the salary).
        """
        return self.salary * 0.20
    def calculate_net_salary(self):
        """
        Calculate the net salary by subtracting tax from the gross salary.
```



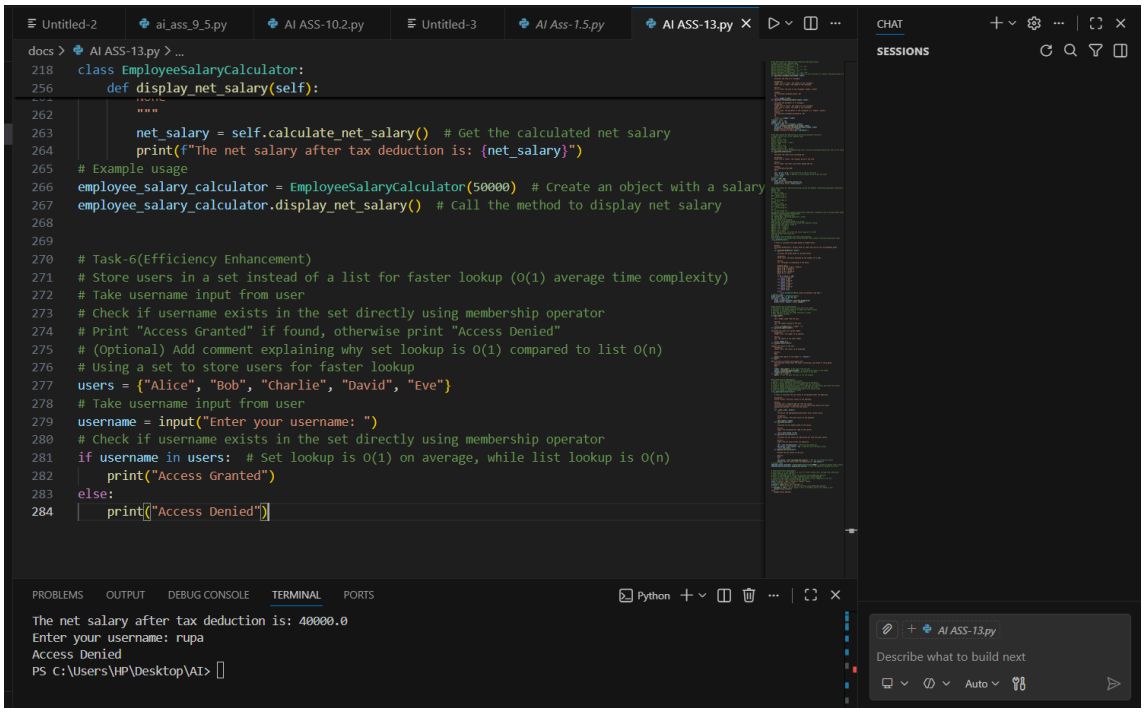
```python
        Calculate the net salary by subtracting tax from the gross salary.

        Returns:
        float: The net salary after tax deduction.
        """
        tax = self.calculate_tax()  # Get the calculated tax
        net_salary = self.salary - tax  # Subtract tax from gross salary
        return net_salary
    def display_net_salary(self):
        """
        Display the net salary to the user.

        Returns:
        None
        """
        net_salary = self.calculate_net_salary()  # Get the calculated net salary
        print(f"The net salary after tax deduction is: {net_salary}")
# Example usage
employee_salary_calculator = EmployeeSalaryCalculator(50000)  # Create an object with a salary value
employee_salary_calculator.display_net_salary()  # Call the method to display net salary
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
The net salary after tax deduction is: 40000.0
```

Task 6 (Optimizing Search Logic)
• Task: Refactor inefficient linear searches using appropriate data structures.
• Focus Areas:
o Time complexity
o Data structure choice
Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
```

print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification

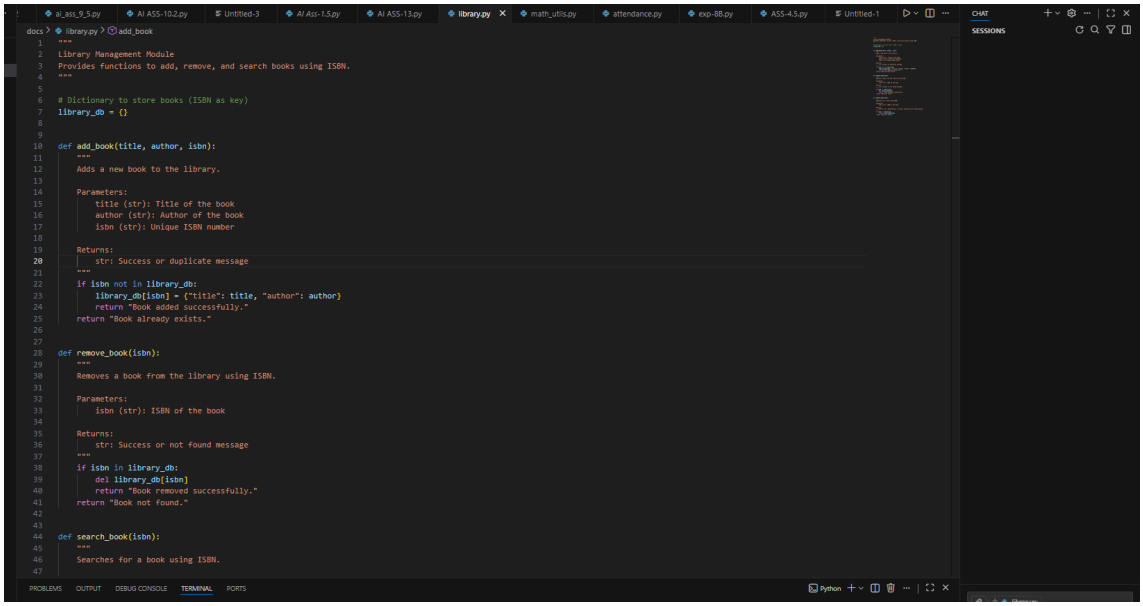


Task 7 – Refactoring the Library Management System
Problem Statement
You are provided with a poorly structured Library Management script that:
Contains repeated conditional logic
Does not use reusable functions
Lacks documentation
Uses print-based procedural execution
Does not follow modular programming principles
Your task is to refactor the code into a proper format
Create a module library.py with functions:
add_book(title, author, isbn)
remove_book(isbn)
search_book(isbn)
Insert triple quotes under each function and let Copilot complete the docstrings.
Generate documentation in the terminal.
Export the documentation in HTML format.
Open the file in a browser.
Given Code

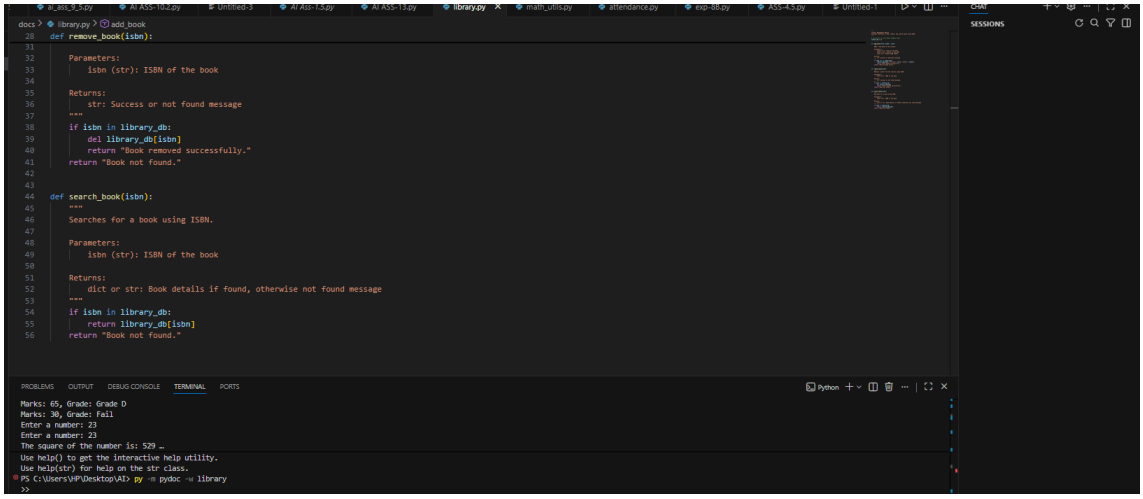
Library Management System (Unstructured Version)

This code needs refactoring into a proper module with documentation

```
library_db = {}

Adding first book

title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")

# Adding second book (duplicate logic)

title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")

# Searching book (repeated logic structure)

isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")

# Removing book (again repeated pattern)

isbn = "101"
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
else:
print("Book not found.")

# Searching again

isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
```



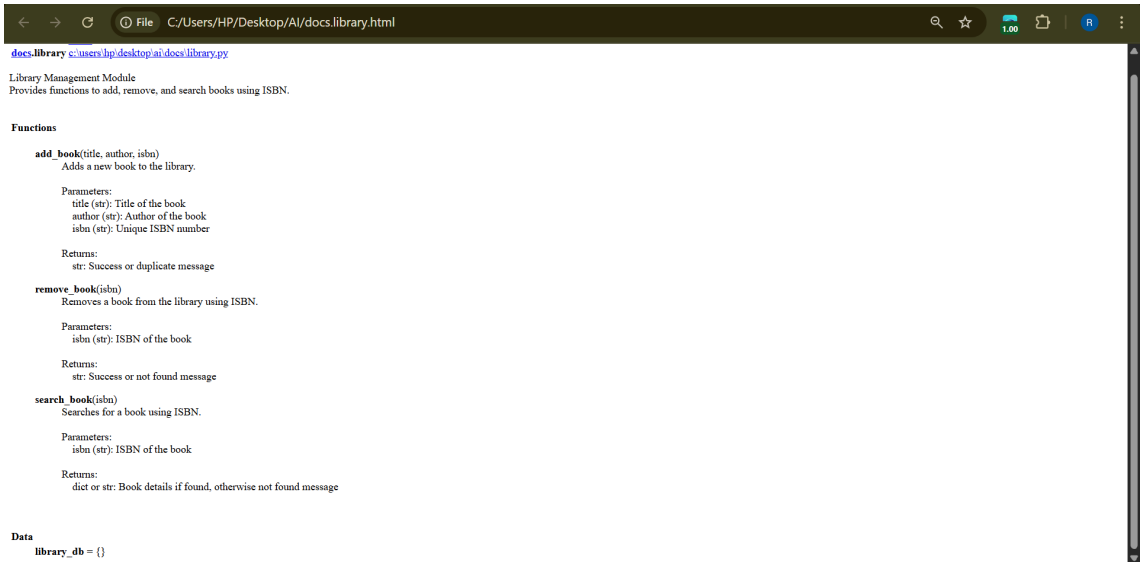
```python
"""
Library Management Module
Provides functions to add, remove, and search books using ISBN.
"""

# Dictionary to store books (ISBN as key)
library_db = {}


def add_book(title, author, isbn):
    """
    Adds a new book to the library.

    Parameters:
        title (str): Title of the book
        author (str): Author of the book
        isbn (str): Unique ISBN number

    Returns:
        str: Success or duplicate message
    """
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        return "Book added successfully."
    return "Book already exists."


def remove_book(isbn):
    """
    Removes a book from the library using ISBN.

    Parameters:
        isbn (str): ISBN of the book

    Returns:
        str: Success or not found message
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    return "Book not found."


def search_book(isbn):
    """
    Searches for a book using ISBN.
```



```python
def remove_book(isbn):

    Parameters:
        isbn (str): ISBN of the book

    Returns:
        str: Success or not found message
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    return "Book not found."


def search_book(isbn):
    """
    Searches for a book using ISBN.

    Parameters:
        isbn (str): ISBN of the book

    Returns:
        dict or str: Book details if found, otherwise not found message
    """
    if isbn in library_db:
        return library_db[isbn]
    return "Book not found."
```

```
Marks: 65, Grade: Grade D
Marks: 30, Grade: Fail
Enter a number: 23
Enter a number: 23
The square of the number is: 529 …
Use help() to get the interactive help utility.
Use help(str) for help on the str class.
PS C:\Users\HP\Desktop\AI> py -m pydoc -w library
>>
```

```
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> start docs.library.html
PS C:\Users\HP\Desktop\AI> start chrome docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> start docs.library.html
PS C:\Users\HP\Desktop\AI> start chrome docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
PS C:\Users\HP\Desktop\AI> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\HP\Desktop\AI> start docs.library.html
PS C:\Users\HP\Desktop\AI> start chrome docs.library.html
```



Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

Global variables.

Inefficient loop.

No functions or modularity.

Task for Students:

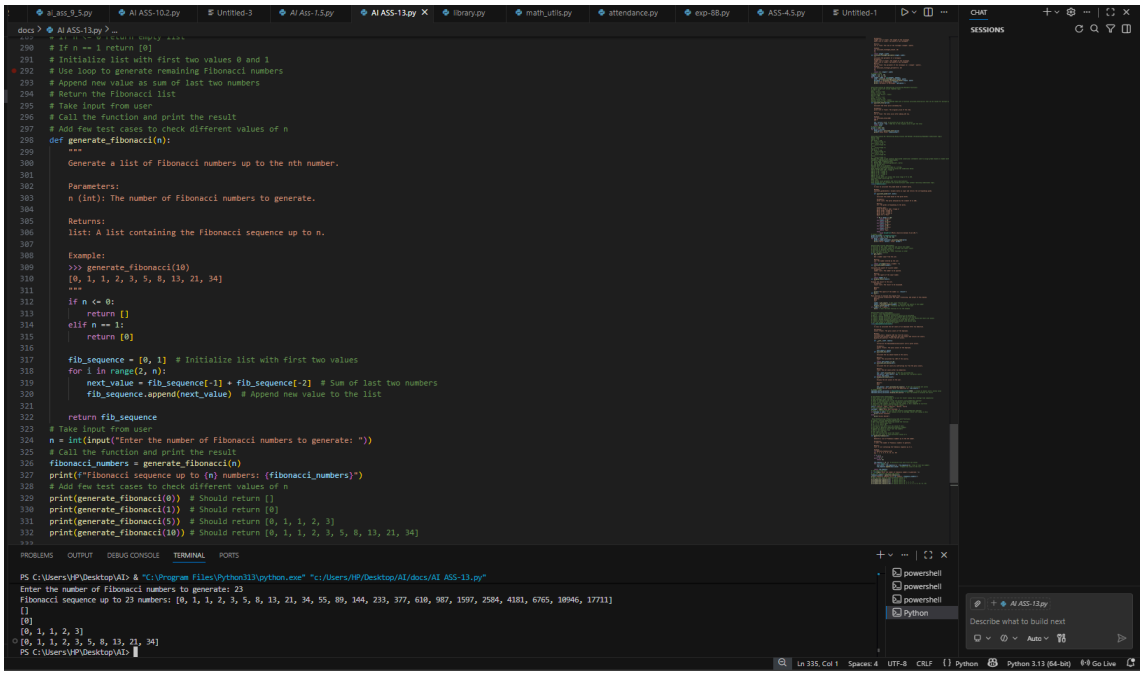
Refactor into a clean reusable function (generate_fibonacci).

Add docstrings and test cases.

Compare AI-refactored vs original.

Bad Code Version:

```
#fibonacci bad version

n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
a=b
b=c
```



Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

Inefficient prime checking.

No functions.

Hardcoded inputs.

Task for Students:

Refactor into is_prime(n) and is_twin_prime(p1, p2).

Add docstrings and optimize.

Generate a list of twin primes in a given range using AI.

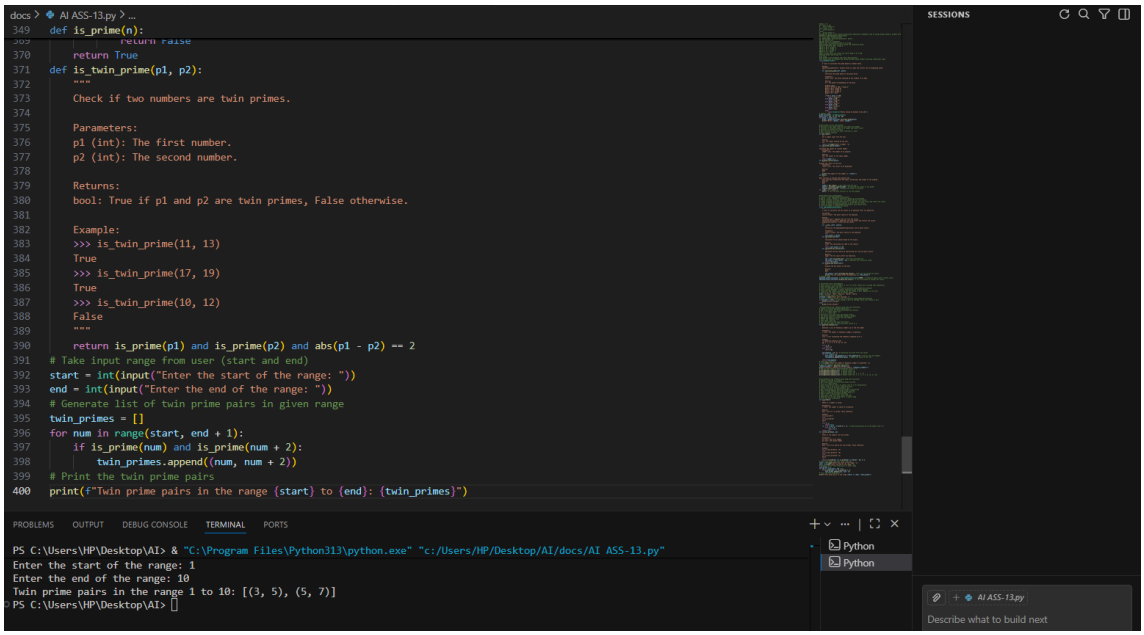
Bad Code Version:

twin primes bad version

```
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
```

print("Twin Primes")
else:
print("Not Twin Primes")

```python
#Task-9(Refactoring – Modularizing Code with Functions)
# Create a function is_prime(n)
# Add triple quotes for docstring inside is_prime
# If n <= 1 return False
# Check divisibility from 2 to square root of n for optimization
# Return True if no divisors found else False
# Create a function is_twin_prime(p1, p2)
# Add triple quotes for docstring inside is_twin_prime
# Check if both numbers are prime using is_prime
# Check if absolute difference between them is 2
# Return True if twin primes else False
# Take input range from user (start and end)
# Generate list of twin prime pairs in given range
# Print the twin prime pairs
def is_prime(n):
    """
    Check if a number is prime.

    Parameters:
    n (int): The number to check for primality.

    Returns:
    bool: True if n is prime, False otherwise.

    Example:
    >>> is_prime(7)
    True
    >>> is_prime(10)
    False
    """
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):  # Check divisibility up to the square root of n
        if n % i == 0:
            return False
    return True
```



```python
def is_prime(n):
    return True
def is_twin_prime(p1, p2):
    """
    Check if two numbers are twin primes.

    Parameters:
    p1 (int): The first number.
    p2 (int): The second number.

    Returns:
    bool: True if p1 and p2 are twin primes, False otherwise.

    Example:
    >>> is_twin_prime(11, 13)
    True
    >>> is_twin_prime(17, 19)
    True
    >>> is_twin_prime(10, 12)
    False
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
# Take input range from user (start and end)
start = int(input("Enter the start of the range: "))
end = int(input("Enter the end of the range: "))
# Generate list of twin prime pairs in given range
twin_primes = []
for num in range(start, end + 1):
    if is_prime(num) and is_prime(num + 2):
        twin_primes.append((num, num + 2))
# Print the twin prime pairs
print(f"Twin prime pairs in the range {start} to {end}: {twin_primes}")
```

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI ASS-13.py"
Enter the start of the range: 1
Enter the end of the range: 10
Twin prime pairs in the range 1 to 10: [(3, 5), (5, 7)]
PS C:\Users\HP\Desktop\AI>
```

Task 10 – Refactoring the Chinese Zodiac Program
Objective
Refactor the given poorly structured Python script into a clean, modular, and reusable implementation. The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not

follow clean coding principles.
Your task is to refactor the code to improve readability, maintainability, and structure.
Chinese Zodiac Cycle (Repeats Every 12 Years)
Rat
Ox
Tiger
Rabbit
Dragon
Snake
Horse
Goat (Sheep)
Monkey
Rooster
Dog
Pig

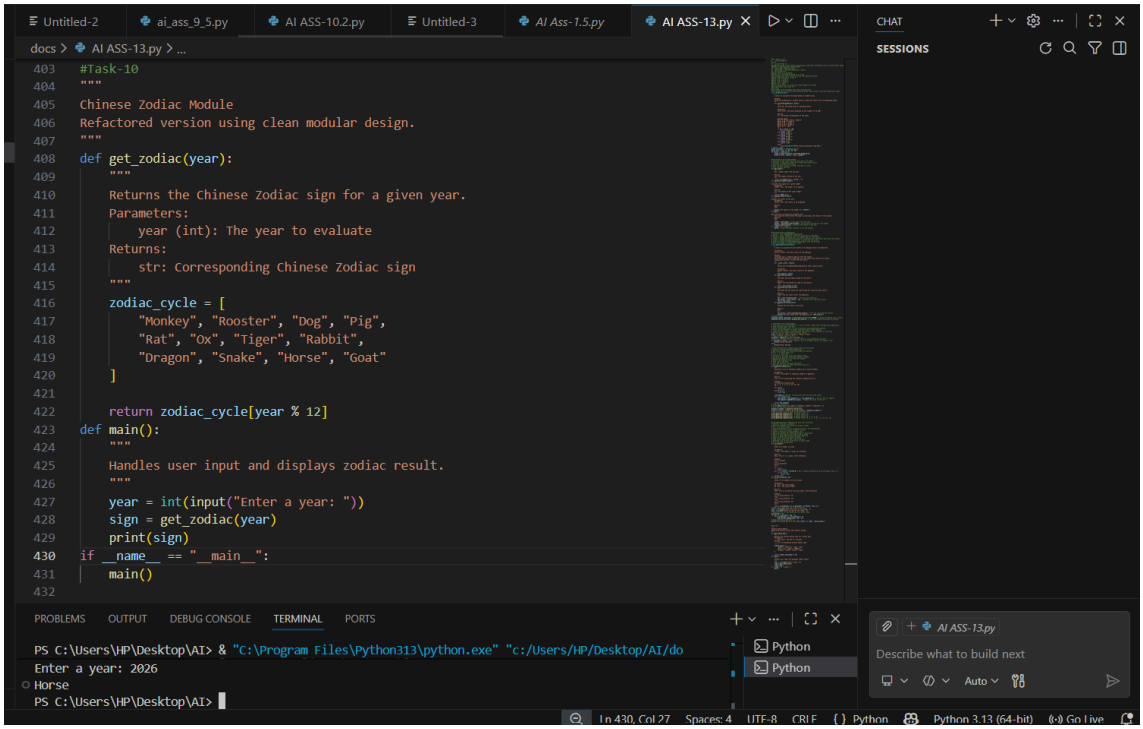
Chinese Zodiac Program(Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))

if year % 12 == 0:
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")
elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:

Create a reusable function: get_zodiac(year)
Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
Add proper docstrings.
Separate input handling from logic.
Improve readability and maintainability.
Ensure output remains correct.



Task 11 – Refactoring the Harshad (Niven) Number Checker
Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.
A Harshad (Niven) number is a number that is divisible by the sum of its digits.
For example:
18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ✅ (Harshad Number)
19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ❌ (Not Harshad)

Problem Statement
The current implementation:
Mixes logic and input handling
Uses redundant variables
Does not use reusable functions properly
Returns print statements instead of boolean values
Lacks documentation
You must refactor the code to follow clean coding principles.

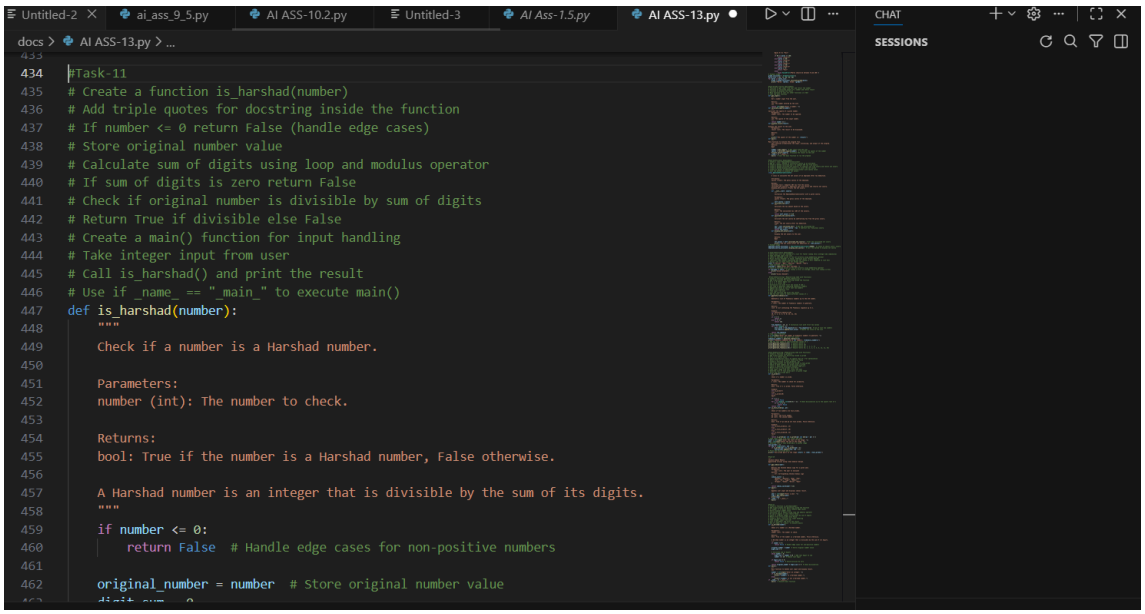
Harshad Number Checker(Unstructured Version)

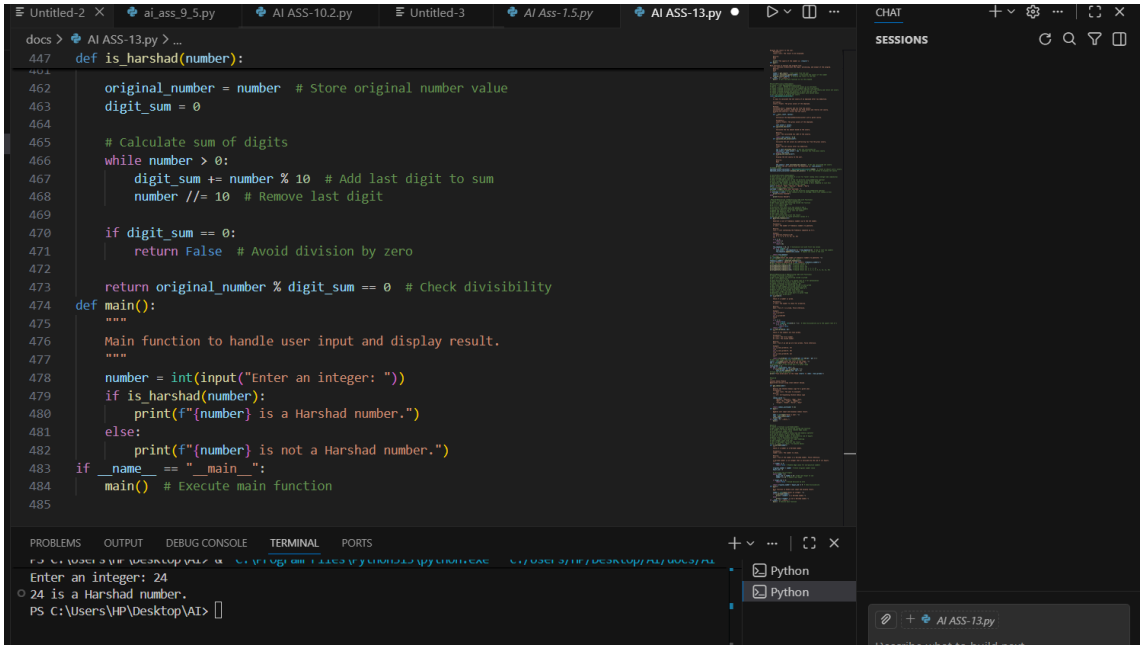
num = int(input("Enter a number: "))

temp = num
sum_digits = 0

while temp > 0:
digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10

if sum_digits != 0:
if num % sum_digits == 0:
print("True")
else:
print("False")
else:
print("False")
You must:
Create a reusable function: is_harshad(number)
The function must:
Accept an integer parameter.
Return True if the number is divisible by the sum of its digits.
Return False otherwise.
Separate user input from core logic.
Add proper docstrings.
Improve readability and maintainability.
Ensure the program handles edge cases (e.g., 0, negative numbers).





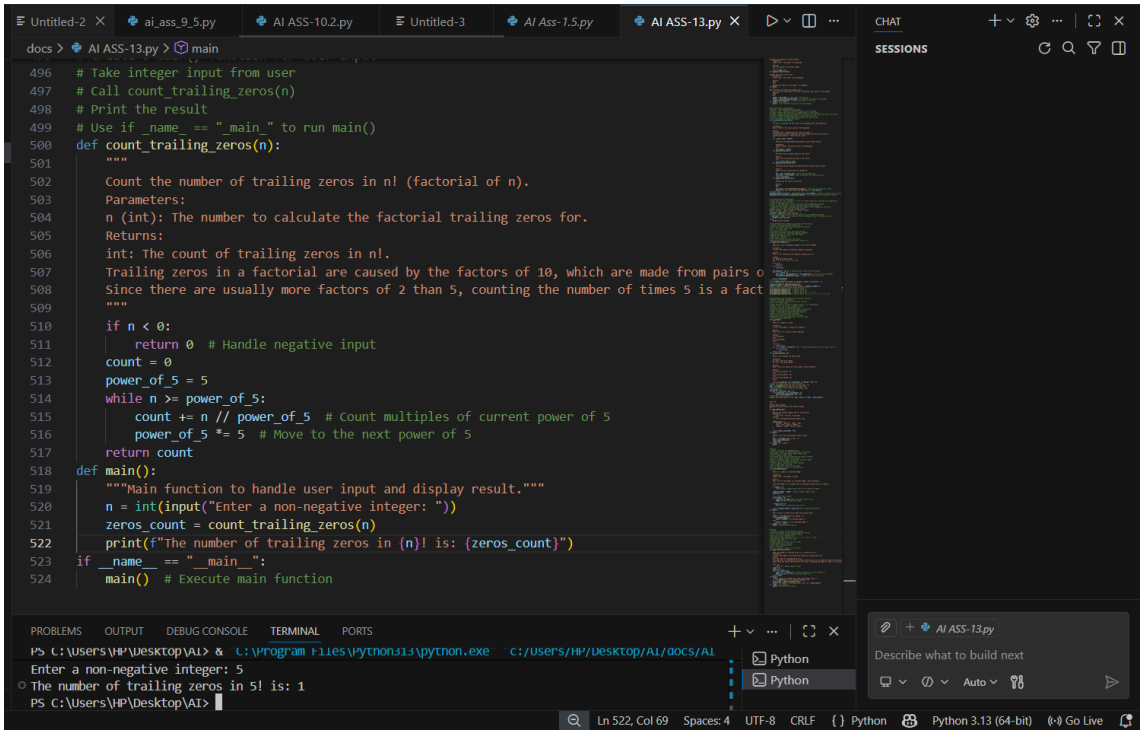
Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.
The program calculates the number of trailing zeros in n! (factorial of n).
Problem Statement
The current implementation:
Calculates the full factorial (inefficient for large n)
Mixes input handling with business logic
Uses print statements instead of return values
Lacks modular structure and documentation
You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros(Unstructured Version)

```
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
count = count + 1
fact = fact // 10
print("Trailing zeros:", count)
```

You must:
Create a reusable function: count_trailing_zeros(n)
The function must:
Accept a non-negative integer n.
Return the number of trailing zeros in n!.
Do NOT compute the full factorial.
Use an optimized mathematical approach (count multiples of 5).
Add proper docstrings.
Separate user interaction from core logic.
Handle edge cases (e.g., negative numbers, zero).

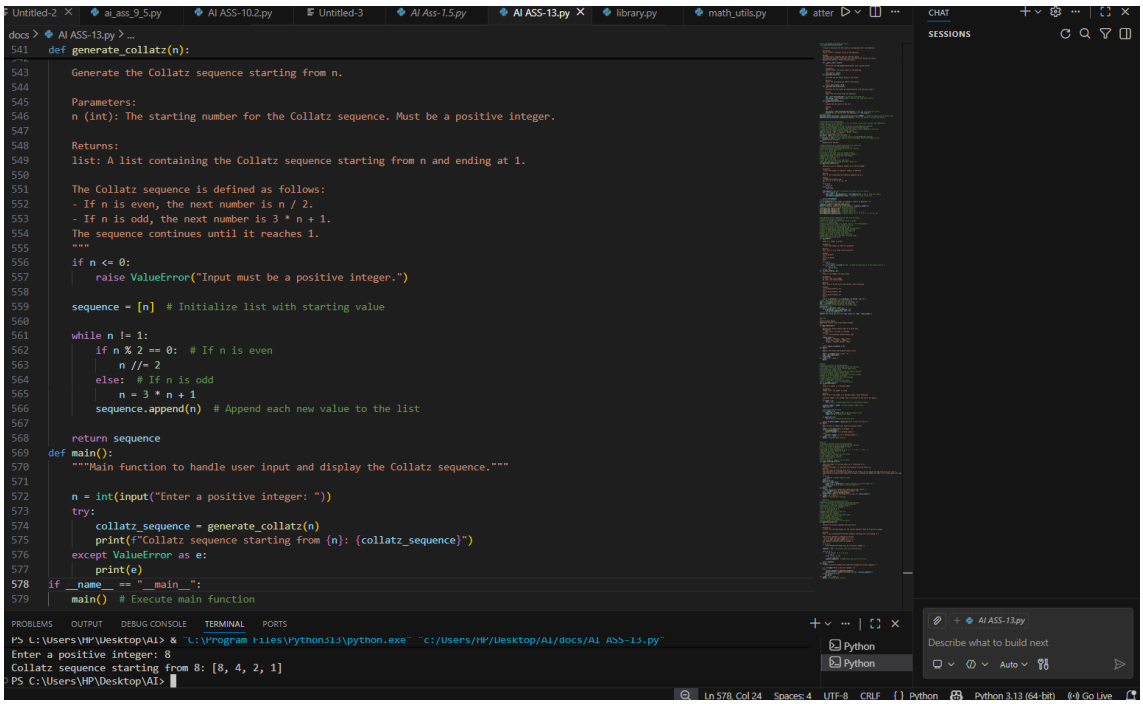



```
    Test Cases Design
```

Task 13 (Collatz Sequence Generator – Test Case Design)
• Function: Generate Collatz sequence until reaching 1.
• Test Cases to Design:
Normal: 6 → [6,3,10,5,16,8,4,2,1]
Edge: 1 → [1]
Negative: -5
Large: 27 (well-known long sequence)
• Requirement: Validate correctness with pytest.
Explanation:

We need to write a function that:
Takes an integer n as input.
Generates the Collatz sequence (also called the 3n+1 sequence).
The rules are:
If n is even → next = n / 2.
If n is odd → next = 3n + 1.
Repeat until we reach 1.
Return the full sequence as a list.

Example
Input: 6
Steps:
6 (even → 6/2 = 3)
3 (odd → 3*3+1 = 10)*
10 (even → 10/2 = 5)
*5 (odd → 3*5+1 = 16)
16 (even → 16/2 = 8)
8 (even → 8/2 = 4)
4 (even → 4/2 = 2)
2 (even → 2/2 = 1)
Output:
[6, 3, 10, 5, 16, 8, 4, 2, 1]



Task 14 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then Fn = Fn-1 + Fn-2)

• Test Cases to Design:

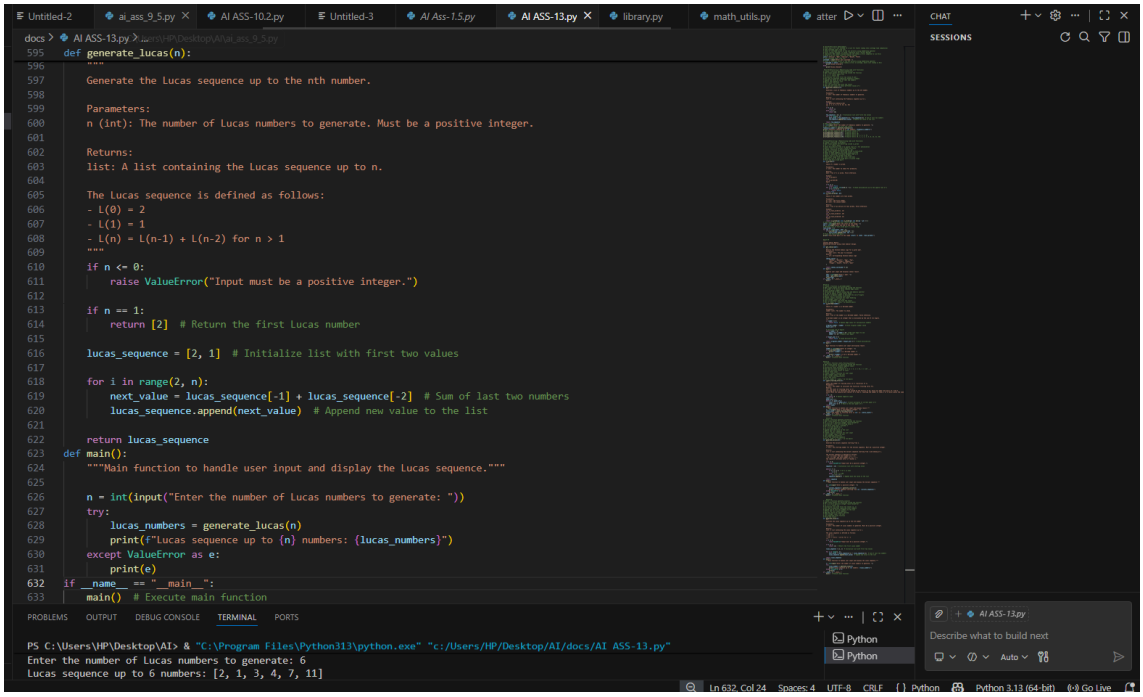
Normal: 5 → [2, 1, 3, 4, 7]

Edge: 1 → [2]

Negative: -5 → Error

Large: 10 (last element = 76).

• Requirement: Validate correctness with pytest.



Task 15 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

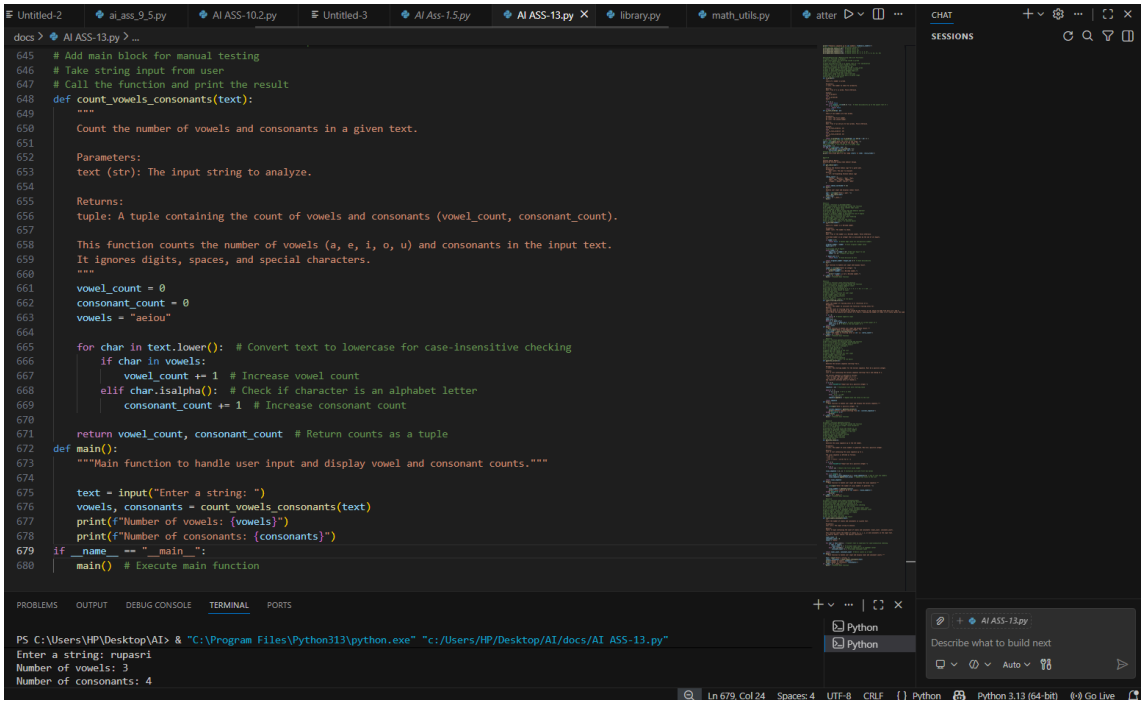
Normal: "hello" → (2,3)

Edge: "" → (0,0)

Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.


Untitled-2
ai_ass_9_5.py
AI ASS-10.2.py
Untitled-3
AI Ass-1.5.py
AI ASS-13.py
library.py
math_utils.py
atter
docs > AI ASS-13.py > ...
# Add main block for manual testing
# Take string input from user
# Call the function and print the result
def count_vowels_consonants(text):
"""
Count the number of vowels and consonants in a given text.
Parameters:
text (str): The input string to analyze.
Returns:
tuple: A tuple containing the count of vowels and consonants (vowel_count, consonant_count).
This function counts the number of vowels (a, e, i, o, u) and consonants in the input text.
It ignores digits, spaces, and special characters.
"""
vowel_count = 0
consonant_count = 0
vowels = "aeiou"
for char in text.lower(): # Convert text to lowercase for case-insensitive checking
if char in vowels:
vowel_count += 1 # Increase vowel count
elif char.isalpha(): # Check if character is an alphabet letter
consonant_count += 1 # Increase consonant count
return vowel_count, consonant_count # Return counts as a tuple
def main():
"""Main function to handle user input and display vowel and consonant counts."""
text = input("Enter a string: ")
vowels, consonants = count_vowels_consonants(text)
print(f"Number of vowels: {vowels}")
print(f"Number of consonants: {consonants}")
if __name__ == "__main__":
main() # Execute main function
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI ASS-13.py"
Enter a string: rupasri
Number of vowels: 3
Number of consonants: 4
Python
Python
CHAT
SESSIONS
AI ASS-13.py
Describe what to build next
Auto
Ln 679, Col 24
Spaces: 4
UTF-8
CRLF
Python
Python 3.13 (64-bit)
Go Live